

# IEORE4004

## Optimization Models and Methods

Yaren Bilge Kaya, PhD

Columbia University  
Department of Industrial Engineering and Operations Research

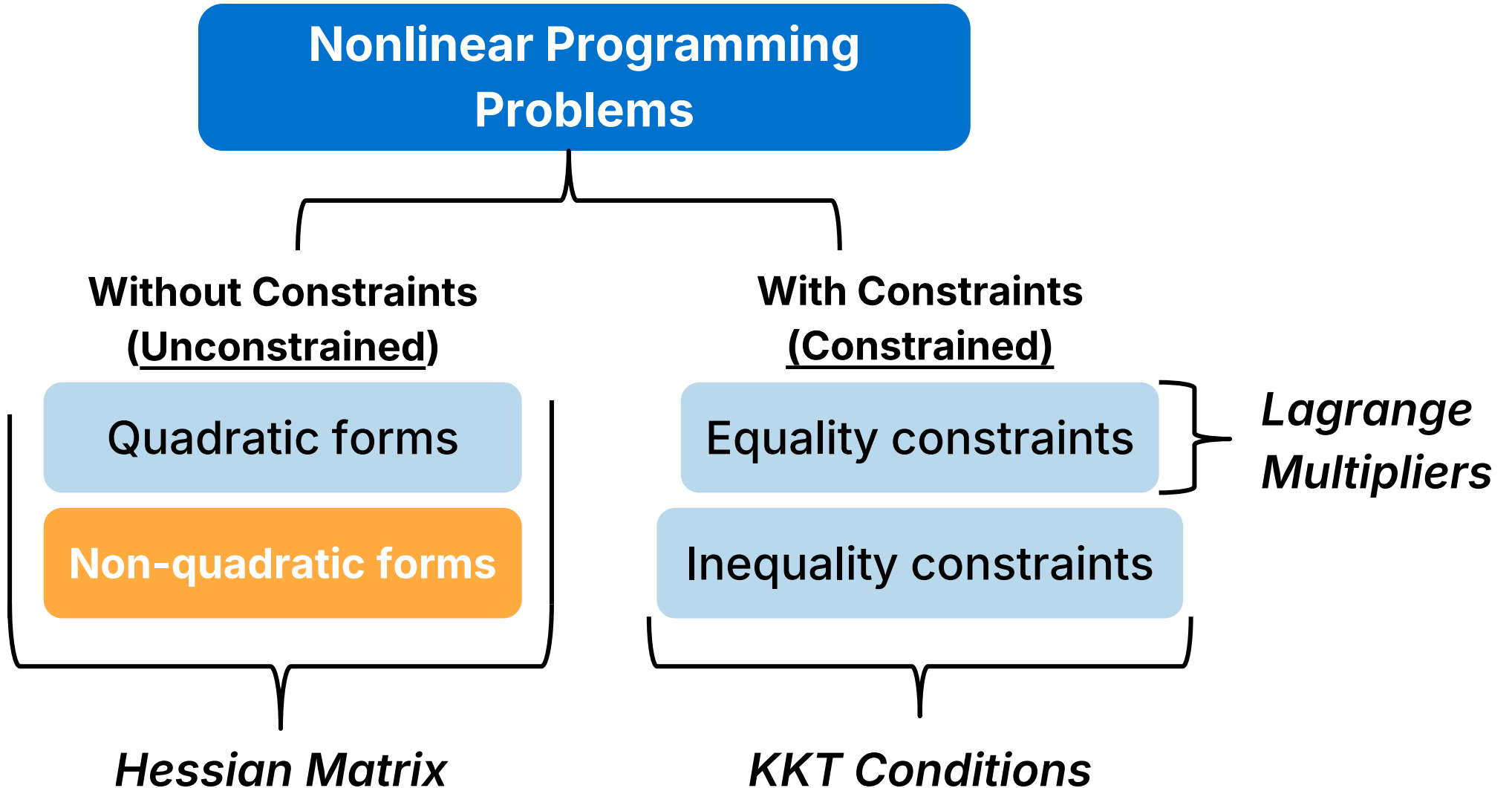


# Outline

---

- Introduction to Non-Quadratic Nonlinear Programming
- Examples
- Introduction to Constrained Nonlinear Programming (With Equalities)

# Types of NLPs



---

# **Introduction to Non-Quadratic Nonlinear Programming**

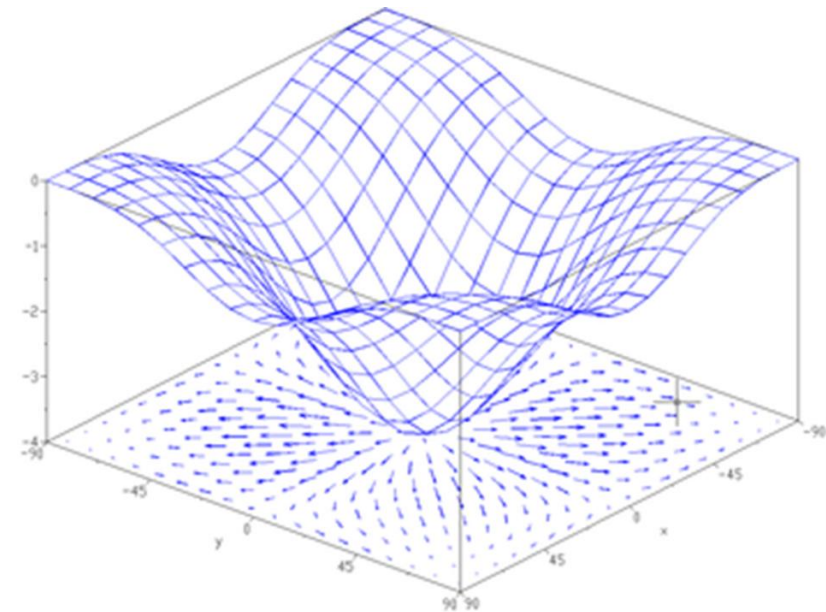
# Gradient of a Function

Consider the twice-differentiable function  $f$ :

The gradient  $\nabla f$  is defined as the vector of first partial derivatives:  $f(\cdot) : \mathbb{R}^n \mapsto \mathbb{R}$

$$\nabla f = \left[ \frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]$$

- The gradient points in the direction of greatest increase
- Thus,  $-\nabla f$  points in the direction of greatest *decrease*
- Points where  $\nabla f = \mathbf{0}$  are known as **stationary points**



<http://en.wikipedia.org/wiki/Gradient>

# Hessian Matrix of a Function

For the same twice-differentiable function  $f(\cdot) : \mathbb{R}^n \mapsto \mathbb{R}$ , consider representing all second partial derivatives

- First derivatives represented with gradient vector is a  $\nabla f$
- Second derivatives require a 2D matrix: carries local **curvature** information
- This is known as the **Hessian matrix  $H$** :

$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

- The **Hessian matrix** is symmetric
- Off-diagonal elements mirror one another
- Can be used in verifying local (global, if convex) optimality conditions for stationary points

# Practice: Finding Gradient, Hessian

---

- Consider the function  $f(x) = 3x_1^2 + 6x_1x_2 + 4x_2^2$
- Take a moment, and calculate its gradient  $\nabla f$
- Now calculate its Hessian  $H$

# Practice: Finding Gradient, Hessian

---

- Consider the function  $f(x) = 3x_1^2 + 6x_1x_2 + 4x_2^2$
- Take a moment, and calculate its gradient  $\nabla f$

$$\nabla f = [6x_1 + 6x_2, 6x_1 + 8x_2]$$

- Now calculate its Hessian  $H$

$$H = \begin{bmatrix} 6 & 6 \\ 6 & 8 \end{bmatrix}$$

# Optimality and the Hessian Matrix

Analyze the Hessian at Critical Points:

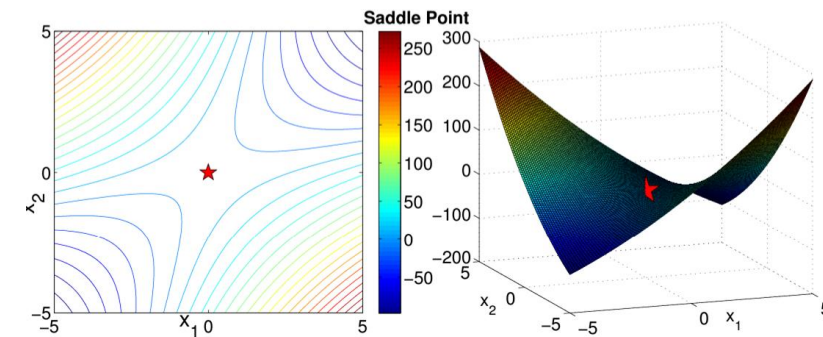
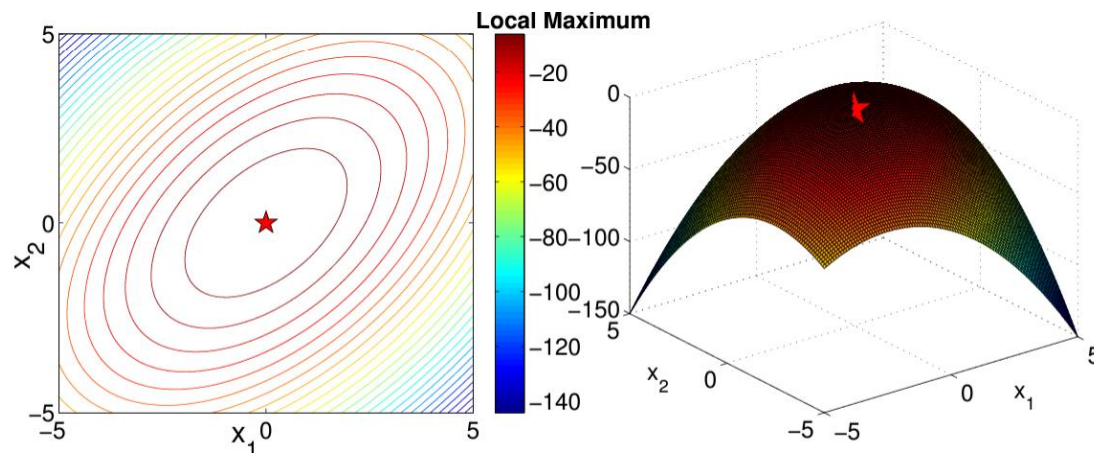
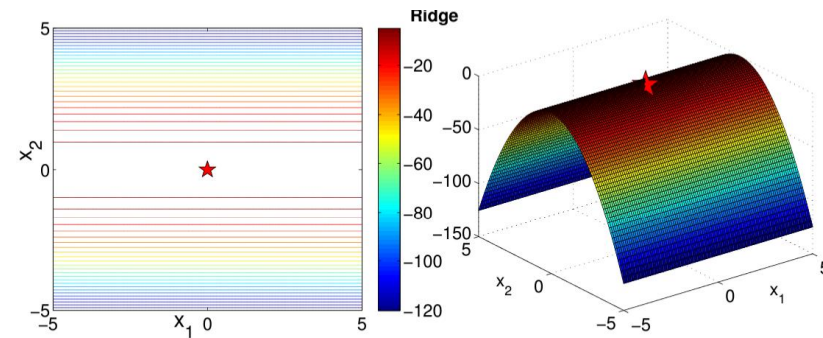
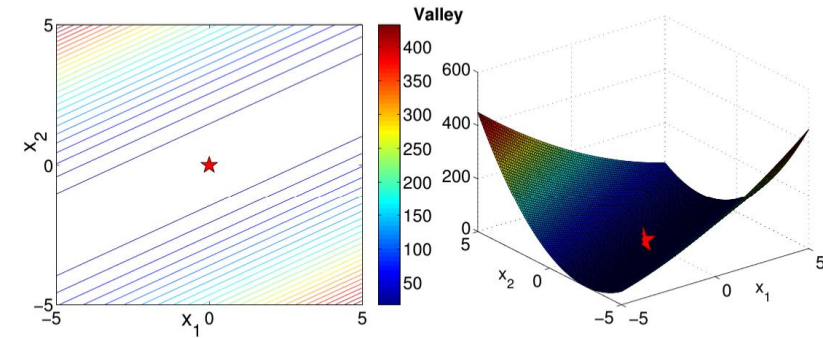
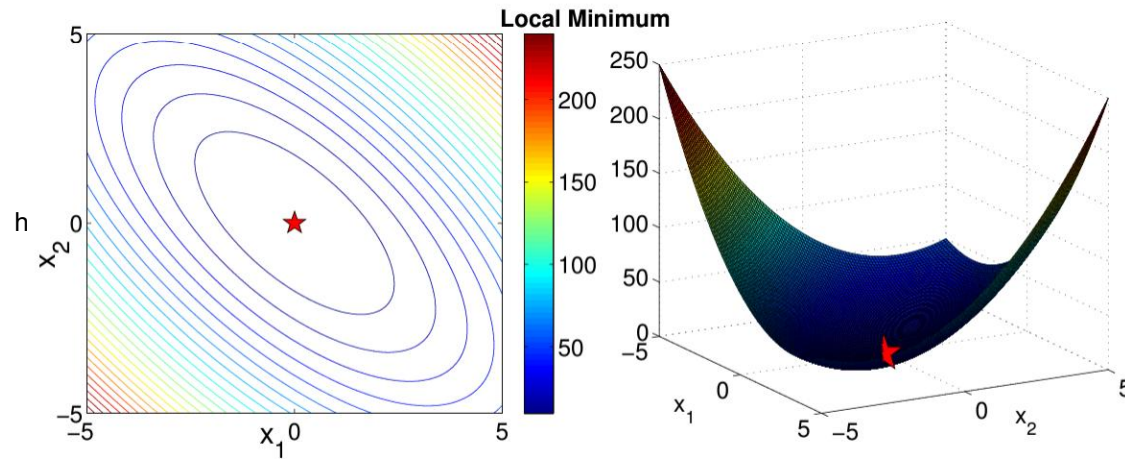
- If the Hessian is **positive definite** at a critical point (all its eigenvalues are positive), the point is a **local minimum**.
- If it is **negative definite** (all eigenvalues are negative), it's a **local maximum**.
- If it is semi-definite (some eigenvalues are zero), the test is inconclusive, and further analysis would be helpful.

Critical point



| Hessian Matrix         | Nature of $x^\dagger$ |
|------------------------|-----------------------|
| positive definite      | local minimizer       |
| negative definite      | local maximizer       |
| positive semi-definite | valley                |
| negative semi-definite | ridge                 |
| indefinite             | saddle point          |

# Optimality and the Hessian Matrix



# Optimality and the Hessian Matrix

---

In many practical **non-quadratic** programming problems, especially those with large dimensions, direct application of the Hessian for finding the optimal solution is computationally expensive or impractical.

In such cases, we use iterative optimization algorithms like:

- Gradient descent,
- Newton's method

These methods utilize the Hessian or approximations of it to iteratively converge to an optimal solution. *(we will cover these soon!)*

# Non-Quadratic Optimization Example using SciPy

Consider this optimization problem:

Minimize  $f(x) = x^3 - 3x + 1$  over the domain of positive real numbers

```
# Define the objective function
def objective_function(x):
    return x**3 - 3*x + 1

# Initial guess
x0 = np.array([0])

# Perform the optimization
result = minimize(objective_function, x0)
```

# SciPy and Unconstrained Optimization

- **minimize (method='CG')**: Conjugate Gradient algorithm, good for mid to large-scale problems.
  - *Uses*: Function values + Gradients
  - *Typical problem size*: 2 to roughly 500 variables.
- **minimize (method='BFGS')**: Broyden–Fletcher–Goldfarb–Shannon algorithm, first-order, Hessian approximation method.
  - *Uses*: Function values + Gradients
  - *Typical problem size*: 2 to roughly 500 variables.
- **minimize (method='Newton-CG')**: Newton-Conjugate Gradient algorithm, uses the Hessian matrix, good for large problems where the Hessian is sparse.
  - *Uses*: Gradient + Hessian or Hessian-vector products
  - *Typical problem size*: Approximately 100 to 10,000 variables
- **minimize (method='Nelder-Mead')**: Simplex algorithm, good for non-differentiable problems.
  - *Uses*: Function values only
  - *Typical problem size*: 1 to 5 variables

# Non-Quadratic Optimization Example using SciPy

Consider this optimization problem:

Minimize  $f(x, y) = (a - x)^2 + b(y - x^2)^2$  where  $a = 1$  and  $b = 100$

```
# Define the Rosenbrock function
def rosenbrock(x):
    a = 1
    b = 100
    return (a - x[0])**2 + b*(x[1] - x[0]**2)**2

# Initial guess
x0 = np.array([2, 2])

# Minimize using BFGS
res_bfgs = minimize(rosenbrock, x0, method='BFGS')
print("BFGS:", res_bfgs)

# Minimize using Nelder-Mead
res_nelder = minimize(rosenbrock, x0, method='Nelder-Mead')
print("Nelder-Mead:", res_nelder)
```

*The Rosenbrock function is a non-convex function used as a performance test problem for optimization algorithms.*

# Comparing BFGS vs. Nelder–Mead on the Rosenbrock

| Metric               | BFGS                 | Nelder–Mead      |
|----------------------|----------------------|------------------|
| Converged?           | Yes                  | Yes              |
| Iterations           | <b>30</b>            | <b>62</b>        |
| Function evaluations | ~105                 | <b>119</b>       |
| Solution found       | [1.0, 1.0]           | [1.0, 1.0]       |
| Accuracy             | Very high (1.89e–11) | Lower (3.42e–09) |

**BFGS reaches the minimum *much faster*, with fewer iterations and higher precision.**

## Which method is better?

For smooth functions like Rosenbrock:

**BFGS is generally better.**

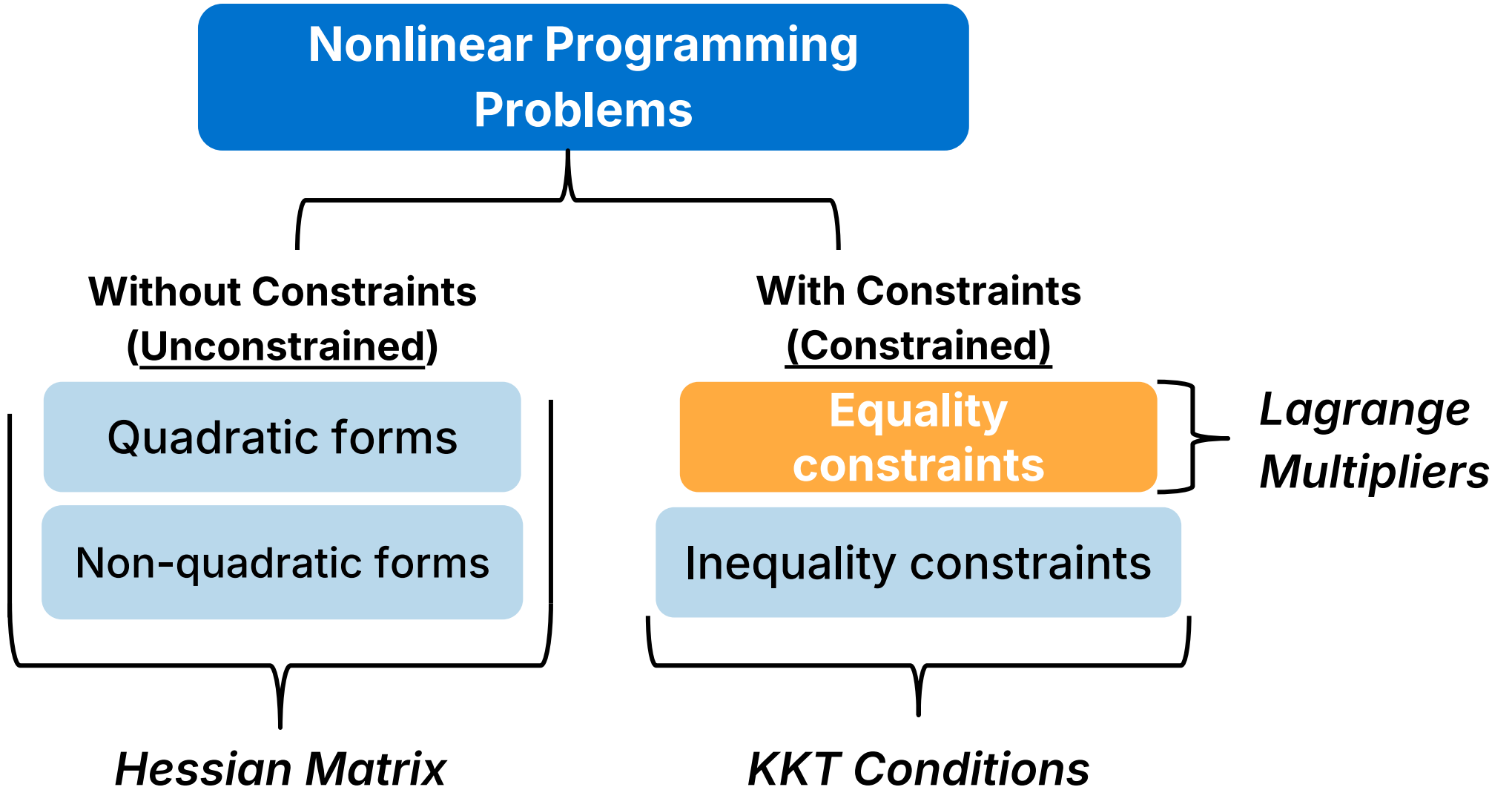
- Faster convergence
- Uses curvature information
- More accurate
- More reliable for high-dimensional problems

### When is Nelder–Mead useful?

- No gradients available
- Function is noisy or discontinuous
- Very small dimensions (2–3 variables)
- When evaluating the function is expensive but derivatives are not available

Nelder–Mead does **not scale!** it becomes slow or fails in high dimensions.

# Types of NLPs



---

# **Introduction to Constrained Nonlinear Programming (with equalities)**

# Optimality Conditions: Equality Case

Consider the equality-constrained nonlinear program:

$$\begin{array}{ll} \min_{x \in \mathbb{R}^n} & f(x) \\ \text{subject to} & g_i(x) = 0, \quad i = 1, \dots, m. \end{array}$$

- Objective is, in general, some nonlinear, differentiable function
- Optimizing over the real variables (i.e., unrestricted)
- There are  $m$  potentially nonlinear constraints

We use the following function, known as the *Lagrangian*:

$$L(x, \lambda) = f(x) + \sum_{i=1}^m \lambda_i g_i(x)$$

# Optimality Conditions: Equality Case

Consider the equality-constrained nonlinear program:

$$\begin{array}{ll} \min_{x \in \mathbb{R}^n} & f(x) \\ \text{subject to} & g_i(x) = 0, \quad i = 1, \dots, m. \end{array}$$

- Objective is, in general, some nonlinear, differentiable function
- Optimizing over the real variables (i.e., unrestricted)
- There are  $m$  potentially nonlinear constraints

We use the following function, known as the **Lagrangian**:

$$L(x, \lambda) = f(x) + \sum_{i=1}^m \lambda_i g_i(x)$$

The condition  $\nabla L = 0$  is a *necessary* condition for optimality

# Optimality Conditions: Equality Case

Again, the Lagrangian function is:  $L(x, \lambda) = f(x) + \sum_{i=1}^m \lambda_i g_i(x)$

- The condition  $\nabla L = 0$  implies:  $\frac{\partial L}{\partial x} = \nabla f(x) + \sum_{i=1}^m \lambda_i \nabla g_i(x) = 0$

$$\text{and } \frac{\partial L}{\partial \lambda_i} = g_i(x) = 0, \quad i = 1, \dots, m$$

# Optimality Conditions: Equality Case

Again, the Lagrangian function is:  $L(x, \lambda) = f(x) + \sum_{i=1}^m \lambda_i g_i(x)$

- The condition  $\nabla L = 0$  implies:  $\frac{\partial L}{\partial x} = \nabla f(x) + \sum_{i=1}^m \lambda_i \nabla g_i(x) = 0$

$$\text{and } \frac{\partial L}{\partial \lambda_i} = g_i(x) = 0, \quad i = 1, \dots, m$$

**The first condition states that the gradient of the objective can be represented as a linear combination of the constraint gradients**

These conditions jointly represent *necessary optimality conditions* for equality constrained nonlinear optimization problems!

# Optimality Conditions: Equality Case

Again, the Lagrangian function is:  $L(x, \lambda) = f(x) + \sum_{i=1}^m \lambda_i g_i(x)$

- The condition  $\nabla L = 0$  implies:  $\frac{\partial L}{\partial x} = \nabla f(x) + \sum_{i=1}^m \lambda_i \nabla g_i(x) = 0$

$$\text{and } \frac{\partial L}{\partial \lambda_i} = g_i(x) = 0, \quad i = 1, \dots, m$$

**The first condition states that the gradient of the objective can be represented as a linear combination of the constraint gradients**

These conditions jointly represent ***necessary optimality conditions*** for equality constrained nonlinear optimization problems!

→ Any point  $x$  satisfying these conditions is a ***stationary point*** for the Lagrangian function

# Practice: Solving Using Lagrangian

Consider the following optimization problem

$$\begin{array}{ll} \min_{x_1, x_2 \in \mathbb{R}} & x_1^2 + x_2^2 \\ \text{subject to} & (x_1 - 2)^2 + x_2^2 = 1. \end{array}$$

- First, form Lagrangian function; then set partial derivatives to zero:

$$L(x_1, x_2, \lambda) = x_1^2 + x_2^2 + \lambda[(x_1 - 2)^2 + x_2^2 - 1]$$

$$\frac{\partial L}{\partial x_1} = 2x_1 + \lambda(2x_1 - 4) = 0 \Rightarrow 2x_1(1 + \lambda) - 4\lambda = 0 \quad (\text{A})$$

$$\frac{\partial L}{\partial x_2} = 2x_2 + \lambda(2x_2) = 0 \Rightarrow 2x_2(1 + \lambda) = 0 \quad (\text{B})$$

$$\frac{\partial L}{\partial \lambda} = (x_1 - 2)^2 + x_2^2 - 1 = 0 \quad (\text{C})$$

## Practice: Trial and Error

---

$$\frac{\partial L}{\partial x_1} = 2x_1 + \lambda(2x_1 - 4) = 0 \Rightarrow 2x_1(1 + \lambda) - 4\lambda = 0 \quad (\text{A})$$

$$\frac{\partial L}{\partial x_2} = 2x_2 + \lambda(2x_2) = 0 \Rightarrow 2x_2(1 + \lambda) = 0 \quad (\text{B})$$

$$\frac{\partial L}{\partial \lambda} = (x_1 - 2)^2 + x_2^2 - 1 = 0 \quad (\text{C})$$

From (B), we got either  $x_2 = 0$  or  $\lambda = -1$  but from (A) we see that if  $\lambda = -1$ , we'll have  $2x_1[1 + (-1)] - 4(-1) = 4 \neq 0$  which is infeasible.

Then set  $x_2 = 0$ , from (C) we have  $(x_1 - 2)^2 - 1 = 0 \Rightarrow x_1 - 2 = \pm 1$  .

## Practice: Trial and Error

---

$$\frac{\partial L}{\partial x_1} = 2x_1 + \lambda(2x_1 - 4) = 0 \Rightarrow 2x_1(1 + \lambda) - 4\lambda = 0 \quad (\text{A})$$

$$\frac{\partial L}{\partial x_2} = 2x_2 + \lambda(2x_2) = 0 \Rightarrow 2x_2(1 + \lambda) = 0 \quad (\text{B})$$

$$\frac{\partial L}{\partial \lambda} = (x_1 - 2)^2 + x_2^2 - 1 = 0 \quad (\text{C})$$

Suppose  $x_1 = 1$ , from (A) we got  $2(1 + \lambda) - 4\lambda = 0 \Rightarrow 2 - 2\lambda = 0 \Rightarrow \lambda = 1$

Suppose  $x_1 = 3$ , from (A) we got  $6(1 + \lambda) - 4\lambda = 0 \Rightarrow 6 + 2\lambda = 0 \Rightarrow \lambda = -3$

By trial and error, we've identified the solution set as  $(1,0,1)$  or  $(3,0,-3)$

# Practice: Solving Using Lagrangian

Consider the following optimization problem:

$$\begin{array}{ll} \min_{x_1, x_2 \in \mathbb{R}} & x_1^2 + x_2^2 \\ \text{subject to} & (x_1 - 2)^2 + x_2^2 = 1. \end{array}$$

- First, form Lagrangian function; then set partial derivatives to zero:

$$L(x_1, x_2, \lambda) = x_1^2 + x_2^2 + \lambda[(x_1 - 2)^2 + x_2^2 - 1]$$

- Setting partial derivative of Lagrangian equal to zero and solving for values (by trial and error!), gives two critical points:
  - The first evaluates to 1:  $x_1 = 1, x_2 = 0, \lambda = 1 \Rightarrow 1$
  - The second evaluates to 9:  $x_1 = 3, x_2 = 0, \lambda = -3 \Rightarrow 9$
  - Clearly, the first is the minimizer!

# Let's Practice Together!

You are given a piece of cardboard measuring  $8\text{m}^2$ . Your task is to design a box with the maximum possible volume using all of this cardboard. How would you determine the length, width, and height of the box to maximize its volume, using the principles of **Lagrangian multipliers**?



# Solution to Example

**Question:** Find the dimensions of the largest box (by volume) if there is 8 m<sup>2</sup> of cardboard.

Use the method of Lagrangian multipliers.

Let  $x$  = length,  $y$  = width, and  $z$  = depth.

Objective: maximize the volume  $V = xyz$

Material constraint:  $2(xy + yz + zx) = 8$

Non-negativity constraint:  $x, y, z \geq 0$

# Solution to Example

**Question:** Find the dimensions of the largest box (by volume) if there is 8 m<sup>2</sup> of cardboard.

Use the method of Lagrangian multipliers.

Let  $x$  = length,  $y$  = width, and  $z$  = depth.

Nonlinear program with equality constraint:

$$\max \quad V = xyz$$

$$\text{s.t.} \quad xy + yz + zx = 4$$

$$x, y, z \geq 0$$

# Solution to Example

---

From the Lagrangian  $L = xyz + \lambda(xy + yz + zx - 4)$ , the necessary condition for optimality is for  $\nabla L = 0$ , that is,

$$\frac{\partial L}{\partial x} = yz + \lambda y + \lambda z = 0 \implies \lambda y + \lambda z = -yz \implies \lambda xy + \lambda xz = -xyz \quad (1)$$

$$\frac{\partial L}{\partial y} = xz + \lambda x + \lambda z = 0 \implies \lambda x + \lambda z = -xz \implies \lambda xy + \lambda yz = -xyz \quad (2)$$

$$\frac{\partial L}{\partial z} = xy + \lambda x + \lambda y = 0 \implies \lambda x + \lambda y = -xy \implies \lambda xz + \lambda yz = -xyz \quad (3)$$

$$\frac{\partial L}{\partial \lambda} = xy + yz + zx - 4 = 0 \quad (4)$$

# Solution to Example

---

Setting equations (1) and (2) equal to each other, we get

$$\lambda(xz - yz) = 0$$

This implies either  $\lambda = 0$  or  $xz = yz \rightarrow x = y$

If  $\lambda = 0$  then from equation (1)

$$\lambda xy + \lambda xy = -xyz \rightarrow xyz = 0$$

Which cannot be optimal because  $x, y, z$  are box dimensions

Thus we have  $x = y$ . Similarly by setting equations (2) and (3) equal to each other, we got  $y = z$ .

# Solution to Example

---

Thus we have  $x = y = z$ , and because of the material constraint  $xy + yz + xz = 4 \Rightarrow 3x^2 = 4 \Rightarrow x = y = z = \sqrt{\frac{4}{3}}$

(Negative root discarded because of non-negativity constraint)

The maximum volume  $V$  is  $V = \left(\sqrt{\frac{4}{3}}\right)^3$

This is a cube with side length =  $\sqrt{\frac{4}{3}}$

# SciPy Example

## Optimizing HVAC\* System for Energy Efficiency

*\*heating, ventilation, and air conditioning*

Let's explore the challenge of optimizing the indoor temperature of the Mudd Engineering Building at Columbia University.

The objective is to minimize energy consumption while ensuring a comfortable environment for students and faculty.



# SciPy Example

---

$$\text{Minimize} \quad E = \sum_{i=1}^{24} \text{rate} \times |T_{\text{indoor}}(i) - T_{\text{outdoor}}(i)|$$

$$\begin{aligned} \text{S.t.} \quad & 22^{\circ}\text{C} - 2^{\circ}\text{C} \leq T_{\text{indoor}}(i) \leq 22^{\circ}\text{C} + 2^{\circ}\text{C} \\ & \forall i \in \{0, 1, \dots, 23\} \end{aligned}$$

$$|T_{\text{indoor}}(i + 1) - T_{\text{indoor}}(i)| \leq \Delta, \quad \forall i \in \{0, 1, \dots, 22\}.$$